

FakeStore

*La arquitectura de esta app no está escrita en un documento.
Está escrita en el build.*

Cuatro requisitos, dos deseables

- ✓

Lista de productos con imagen, título y precio — con estados de carga y de error

catalog/ui/catalog • 13 tests
- ✓

Pantalla de detalle con imagen grande, descripción, precio y categoría

catalog/ui/detail
- ✓

Favoritos que sobreviven al reinicio

favorites/ • 3 módulos
- ✓

Caché offline con indicador visual de datos guardados

Room es la SSOT • FsStatusBanner

- ✦

Deseable — construir mi propia API

DISEÑADO, NO CONSTRUIDO
- ✦

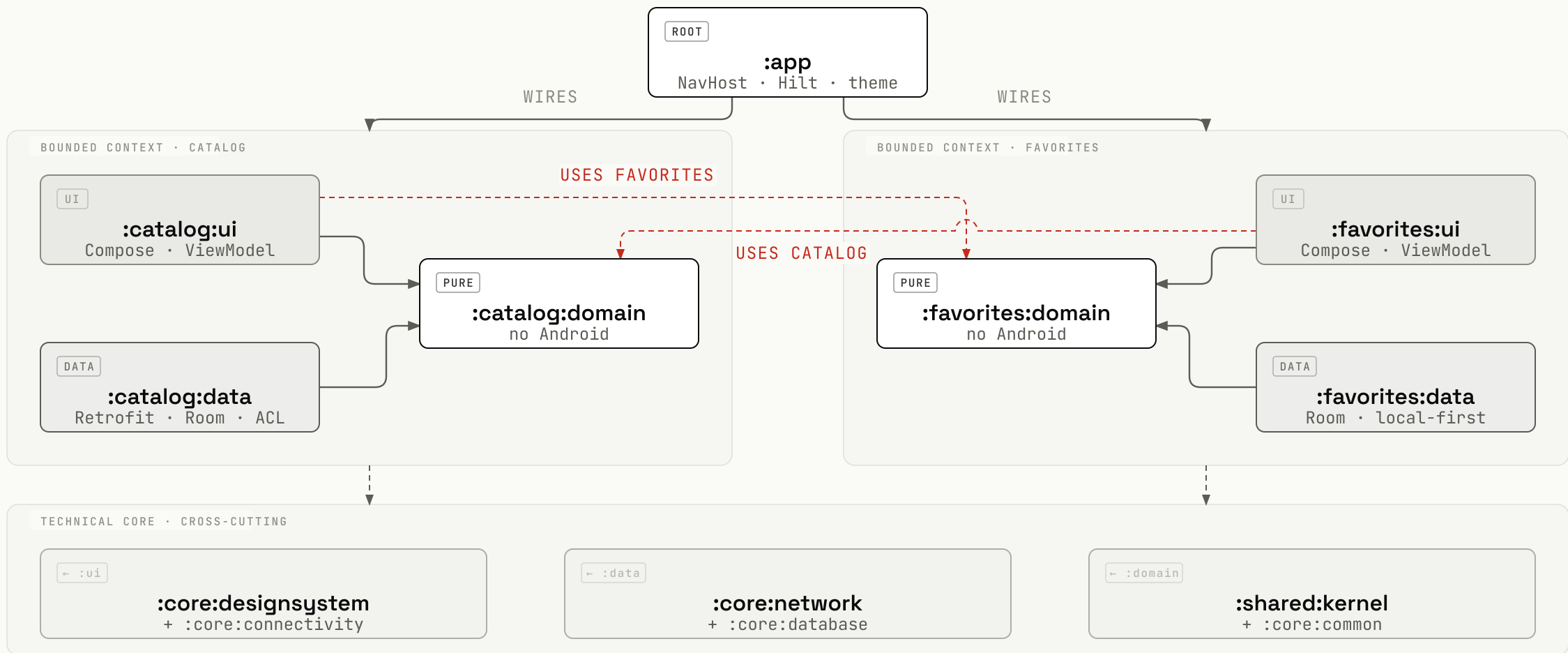
Deseable — “si logras hacerlo con server-driven UI, entonces eres un rockstar”

DISEÑADO, NO CONSTRUIDO

Quien evalúa debería poder distinguir lo que quedó fuera de tiempo de lo que quedó fuera de pensamiento.

Layers are Gradle modules

13 production modules · the purity of :domain is a classpath property, not a code-review convention



LEGEND



UI · presentation



PURE · Kotlin/JVM



DATA · Retrofit · Room



TECHNICAL core



--- cross-context edge

Una capa es un módulo de Gradle, no una carpeta

El módulo de dominio del catálogo no tiene Retrofit ni Room en su classpath.

No es que esté mal visto importarlos: **no compila**.

¿Imposible? No: se arregla agregando la dependencia al build. Lo que no se puede es cometer la violación **desde el código** — hay que abrir un archivo de nueve líneas, que se toca dos veces al año, donde el diff dice textual que estás metiendo HTTP en el dominio.

No es una pared infranqueable. Es una pared que no cruzás distraído, y que deja marca cuando la cruzás.

```
CATALOG/DOMAIN/.../USECASE/REFRESHCATALOG.KT
```

```
import retrofit2.Retrofit
```

e: Unresolved reference 'retrofit2'

se arregla editando el build del módulo

Y ACÁ SÍ ES ABSOLUTO

```
import android.content.Context
```

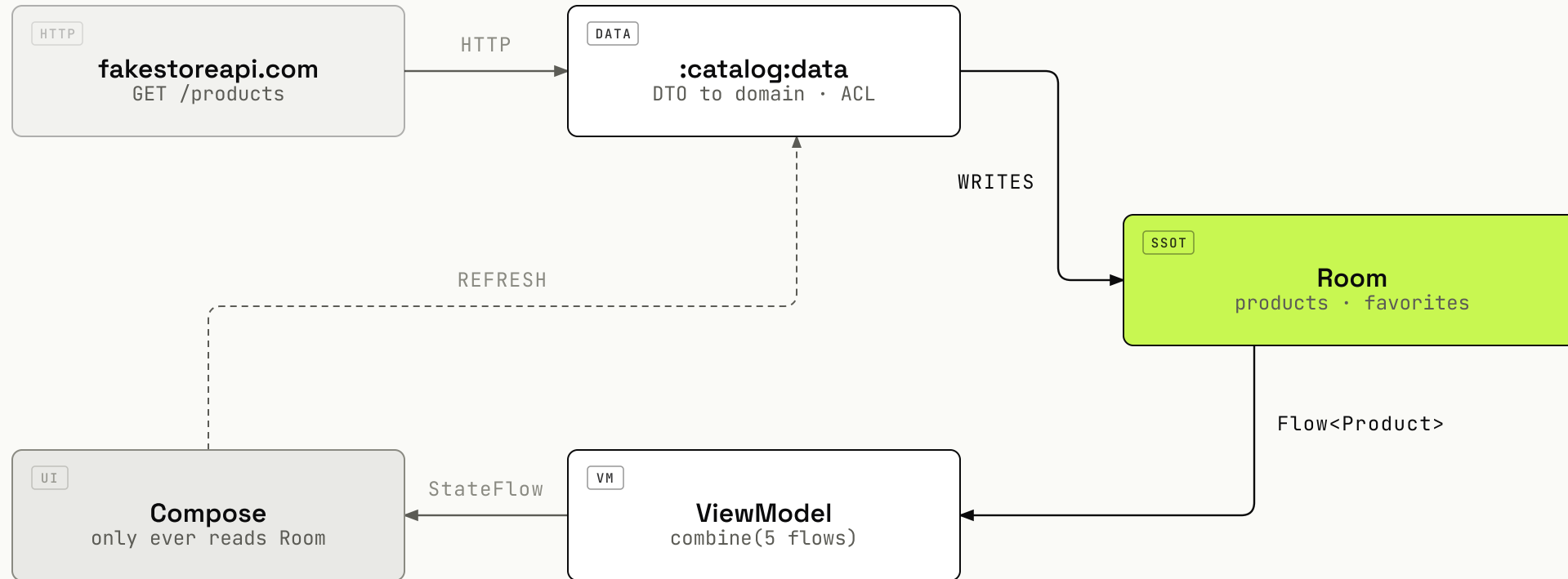
el módulo es kotlin.jvm, no android.library:
no hay android.jar en el classpath, y eso
no se arregla con ninguna dependencia

LO QUE PAGA

Catorce módulos para dos pantallas. Sincronizaciones más lentas y más fricción de Gradle. En un equipo chico, o en un producto todavía exploratorio, elegiría *vertical slices*, con las capas como paquetes.

Room is the single source of truth

Exactly one arrow into the database and one arrow out · that is why no offline mode exists in the code

**LEGEND**

network · writes only



single source of truth

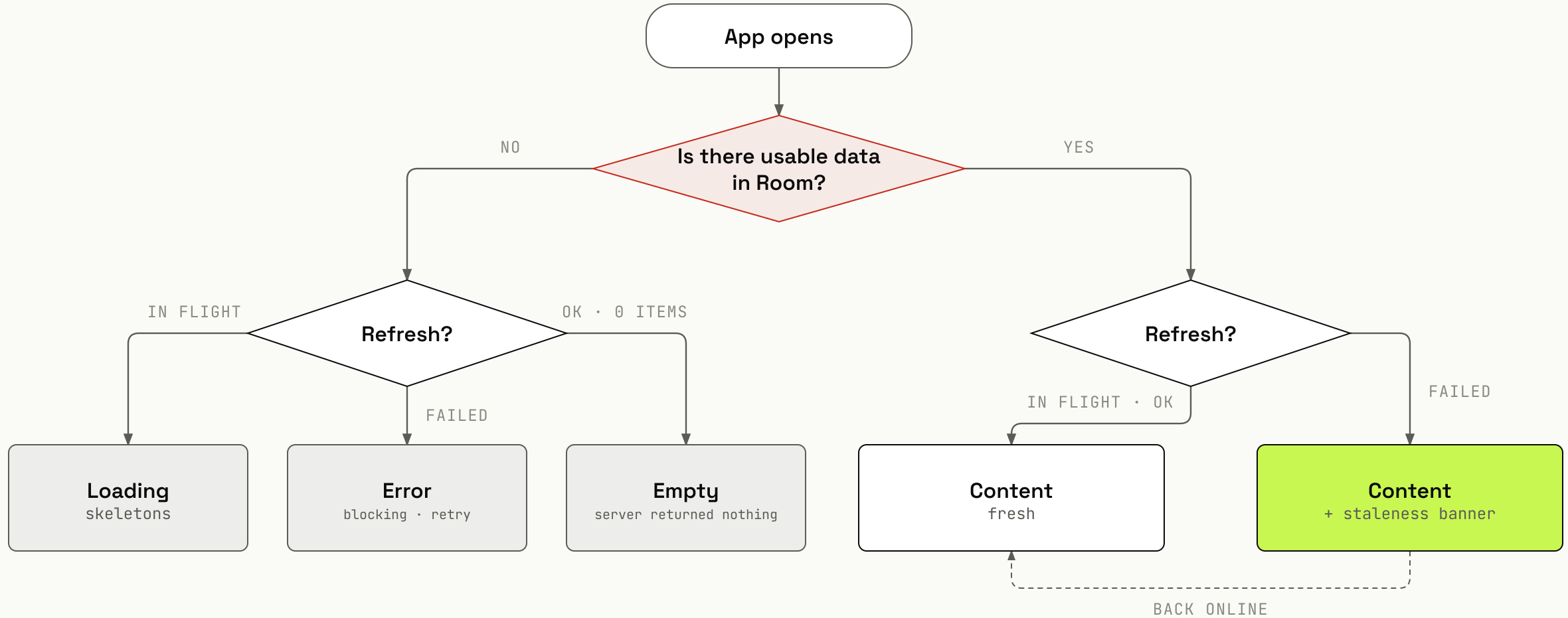


presentation · reads only

--- refresh the UI triggers

The cache decides whether an error is blocking

The axis is not “did the network fail?” • a failed refresh must never hide data the user already had

**LEGEND**

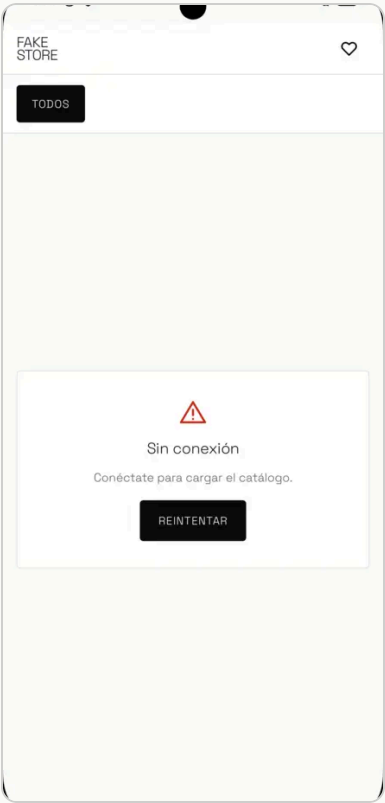
the axis that governs everything



a failed refresh never hides data

--- automatic retry when the network returns

El reintento es automático



SIN CACHÉ
modo avión → red → carga sola

Sin caché y sin red, la pantalla de error es lo correcto: no hay nada que mostrar.

Con caché, el contenido **se queda**. Aparece una banda con la antigüedad de los datos, y un aviso que no bloquea nada.

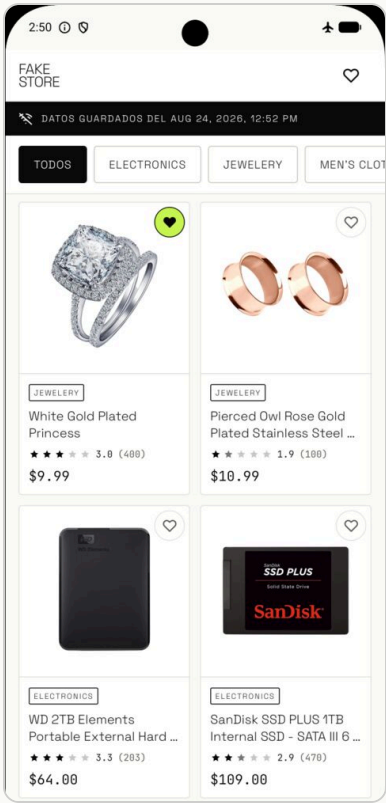
Un fallo de red nunca debe tapar datos que el usuario ya tenía.

Y cuando vuelve la conexión, la app reintenta sola: la banda se cierra sin que nadie toque Reintentar.

5 estados · 13 tests

cada rama es un test de presentación con tiempo virtual, no una verificación manual

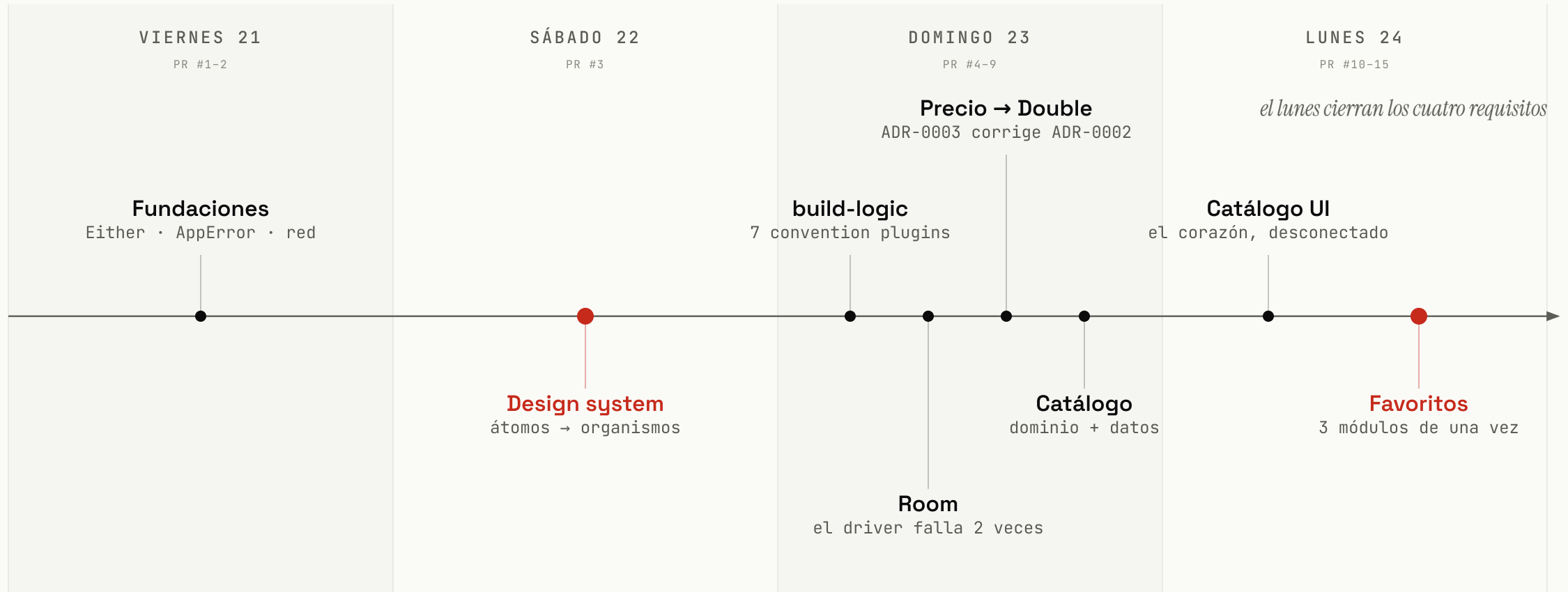
CAPTURAS REALES · TRES TOMAS CONSECUTIVAS DE LA MISMA CORRIDA



CON CACHÉ
el contenido se queda + banda

Cuatro días, quince pull requests

126 commits • ninguna etapa reescribió la anterior: crecer fue agregar módulos

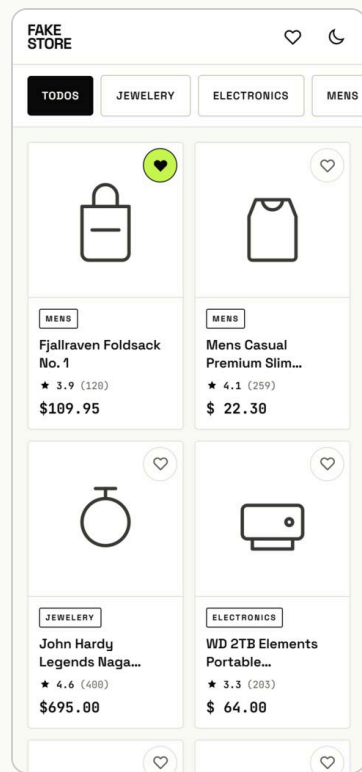


LEYENDA

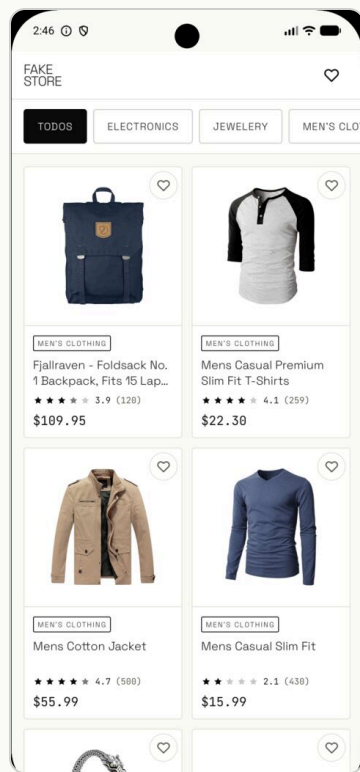
● bloque de trabajo, un pull request

● los dos hitos que prueban la flexibilidad de la arquitectura

El design system se construyó antes que la primera pantalla



ESPECIFICADO
1 de 13 láminas



ENTREGADO
el build

Trece láminas dibujadas antes del primer componente de código.

Atomic design se inventó para la web, donde una página es un archivo. En Android hay módulos, ViewModels y navegación, así que sus cinco capas hubo que **traducirlas, no copiarlas**.

El corte que importa está entre el organismo y la plantilla: todo lo que está por encima se puede previsualizar sin lanzar la app.

átomo

molécula

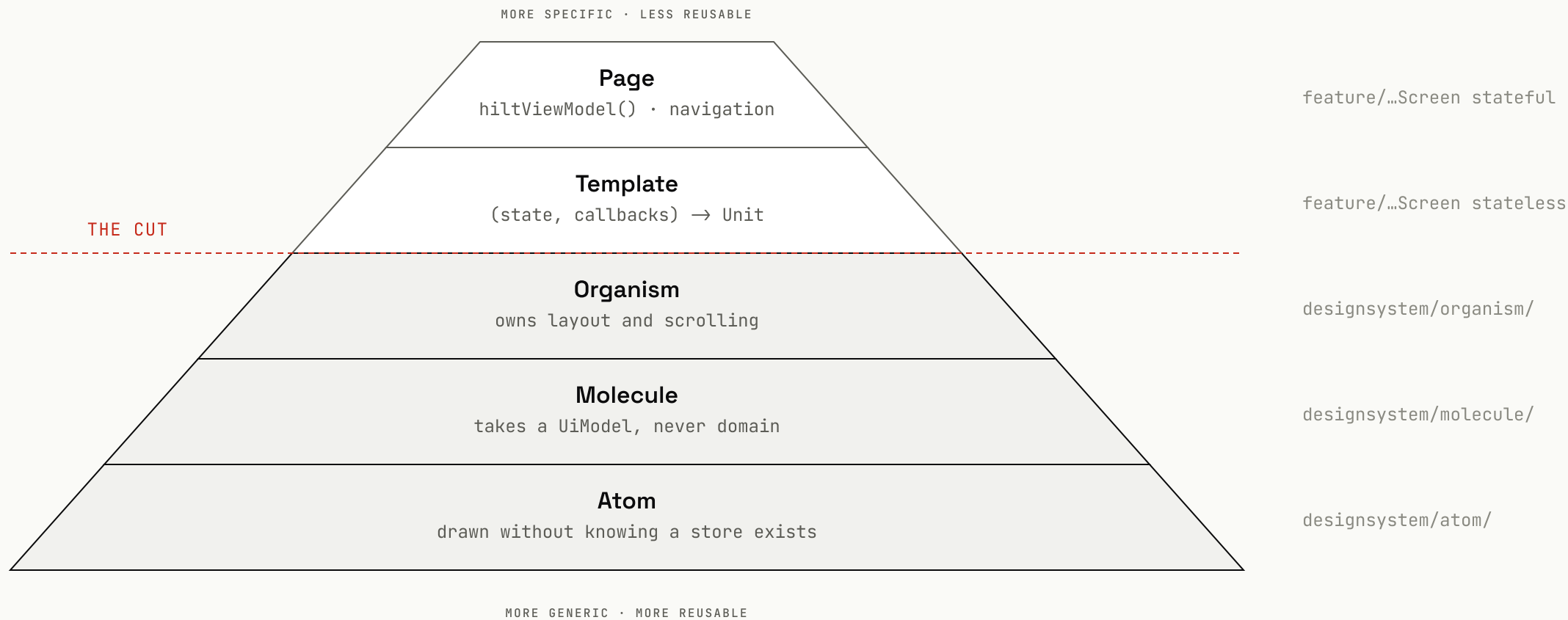
organismo

plantilla

página

From atom to screen

Atomic design translated to Gradle and Compose · the cut that matters is between organism and template



LEGEND



needs the app running



previewable without launching the app

--- the cut that matters

Se borra un contexto entero; no se desarma una app

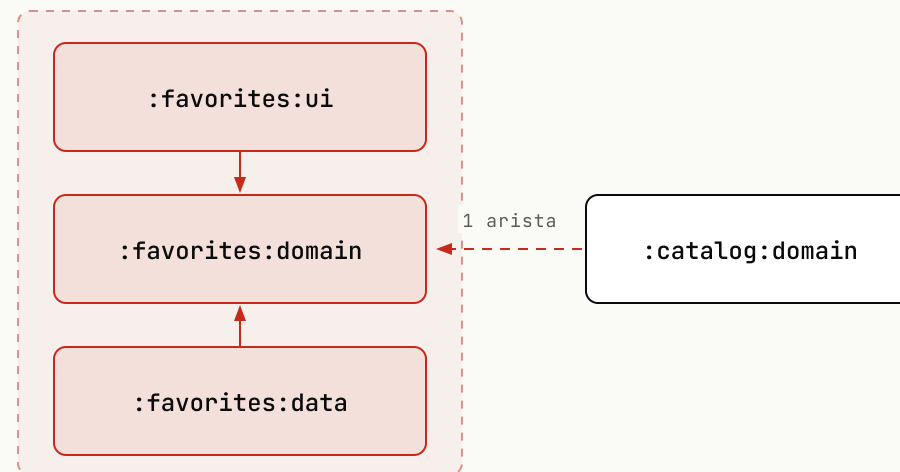
Durante dos bloques enteros, esta app tuvo un corazón dibujado que no hacía absolutamente nada. Fue a propósito: simularlo con estado en memoria habría hecho *parecer* cumplido justo el requisito que pide persistir entre reinicios.

Cuando llegó el contexto completo —dominio, datos y su propia pantalla— el catálogo y el detalle **no se tocaron**. Se agregó una línea en el ViewModel que cruza los dos flujos.

```
combine/catalogFlow, favoriteIdsFlow)
```

Y ahora demos vuelta el argumento. Si mañana el negocio decide que favoritos no va, se borran tres módulos y sus tres aristas, y el dominio del catálogo ni se entera.

FAVORITES · CONTEXTO COMPLETO



borrar favoritos = 3 módulos + 3 aristas de Gradle

:catalog:domain y :catalog:data: cero referencias · sin llave foránea
la presentación pierde un campo por estado · la tabla, una migración

Qué gané, qué pagué, cuándo elegiría distinto

ocho acá • doce en arquitectura • van cuatro

Una decisión sin costo declarado es una decisión que no se pensó.

LA DECISIÓN	GANÉ	PAGUÉ / CUÁNDO ELEGIRÍA DISTINTO
La capa es un módulo de Gradle	La regla la impone el compilador, no la revisión de código.	Catorce módulos para dos pantallas. En un equipo chico: <i>vertical slices</i> .
No hay interceptor de mocks	El arranque en frío —sin caché y sin red— fue un caso real que hubo que diseñar, en vez de un escenario que el mock esconde.	La primera apertura necesita conexión. Contra una API inestable o privada, el mock sí valdría la pena.
El precio es un número simple, no un tipo monetario	La API no manda campo de moneda y en la app no hay una sola operación aritmética. Sin aritmética no hay error de redondeo.	El día que haya carrito, totales o impuestos hay que volver a modelarlo — y ese disparador ya está escrito.
El botón de favorito ignora lo que la pantalla le dice	Una transacción abre, lee la tabla y decide. La autoridad es la base, no lo que la pantalla tenía dibujado: dos taps rápidos no desincronizan nada.	Un parámetro que se ignora y que, a primera vista, parece un descuido.

El servidor manda estructura y referencias, nunca contenido

PAYLOAD DE PANTALLA · CONTRACT VERSION 1

```
{ "screenId": "product_list", "contractVersion": 1,
  "root": { "type": "lazy_list", "id": "list", "source": "catalog",
    "itemTemplate": { "type": "product_card", "id": "card",
      "image": "{product.image}", "title": "{product.title}",
      "price": "{product.price}",
      "trailing": { "type": "favorite_toggle",
        "productId": "{product.id}" } } } }
```

el cliente resuelve esas referencias contra el dominio
que ya tiene cacheado en Room

El servidor decide cómo se ve la pantalla y la app la dibuja. El valor de negocio es directo: cambiar el orden de una pantalla, o lanzar una campaña, **sin pasar por una publicación en la tienda.**

Hecho de la forma habitual choca de frente con el offline-first: el servidor manda la pantalla ya rellena, y eso obliga a cachear pantallas renderizadas — justo lo que rompe la fuente única de verdad.

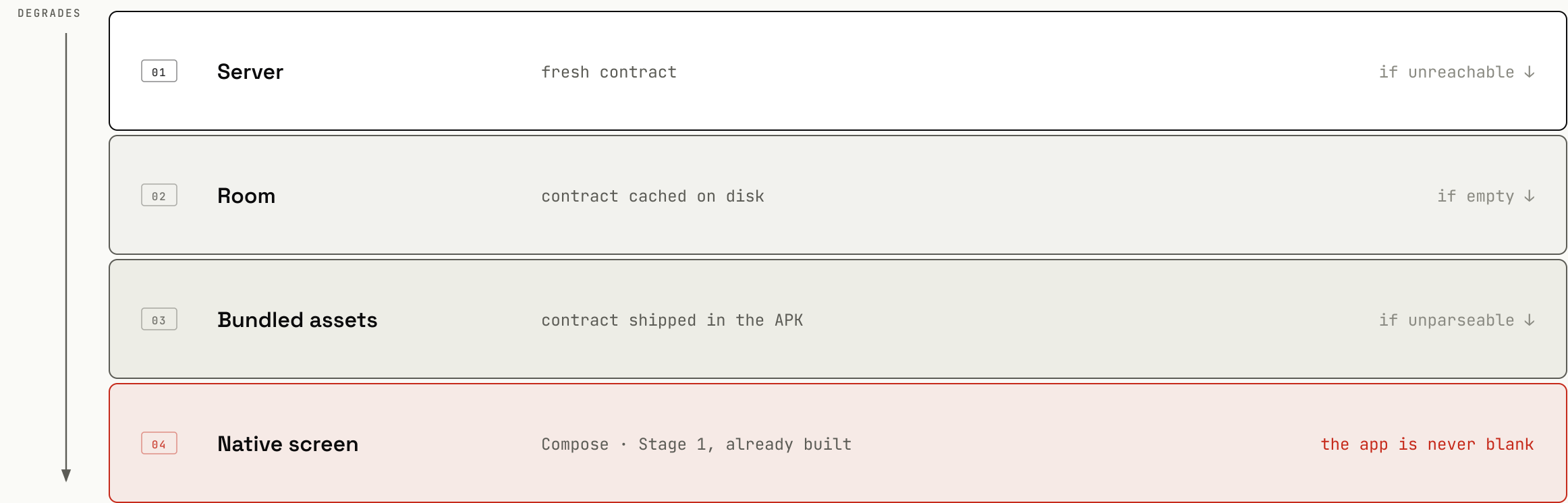
El payload no dice el título del producto. Dice «acá va el título del producto».

Layout cacheado más datos cacheados significa que la app puede renderizar una pantalla dirigida por el servidor **estando en modo avión.**

LO QUE PAGA	Complejidad en el cliente, que tiene que resolver esas referencias.
-------------	---

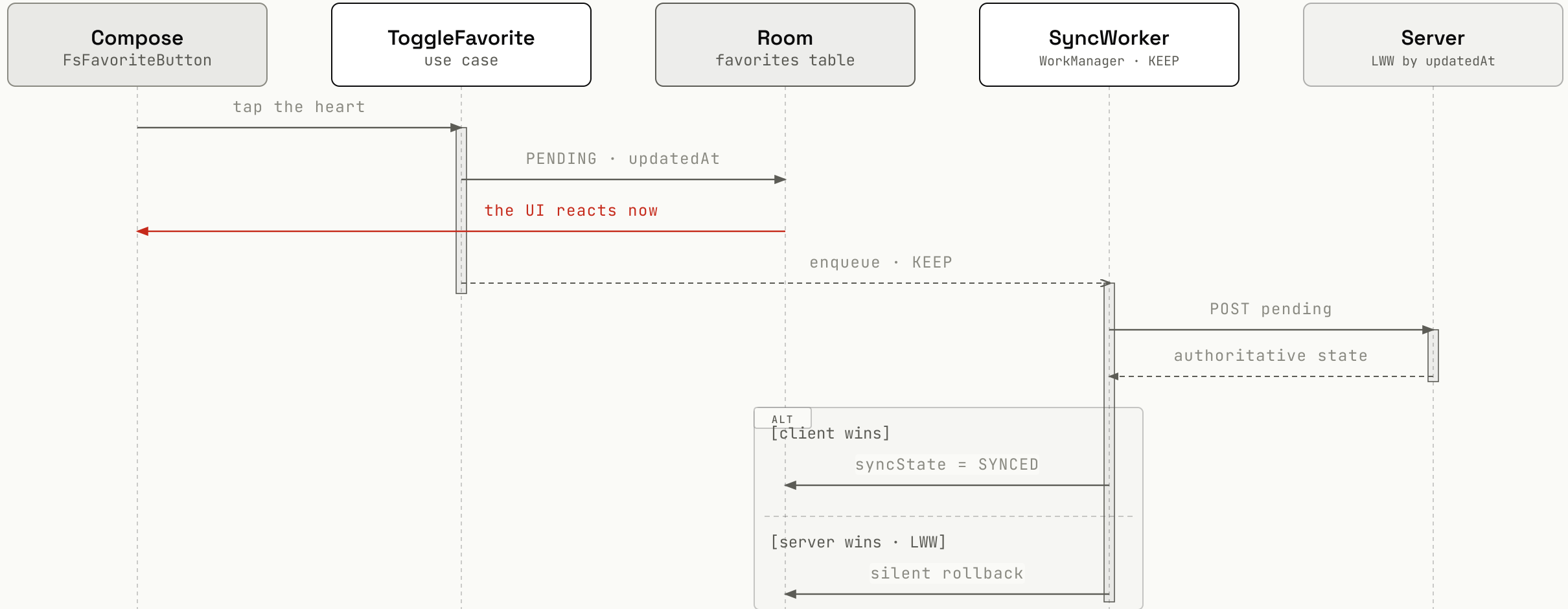
The SDUI fallback ladder

The real risk of SDUI is the server leaving you with no UI · each level is used only when the one above fails



The favorite toggle does not wait for the network

Optimistic local write · deferred sync · last-write-wins with silent rollback



LEGEND

— the UI reacts before the network

--- return or async enqueue

□ local authority · Room decides

Está todo escrito en el repositorio

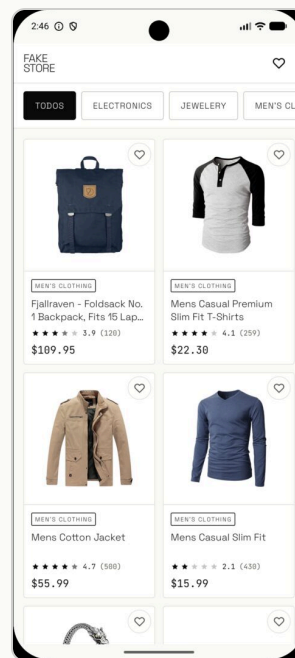
El alcance quedó por encima del mínimo, y fue deliberado. La app que cumple los cuatro requisitos se sostiene sola; todo el resto existe para mostrar cómo pienso un producto que tiene que crecer.

LO QUE FALTA, Y LO SÉ

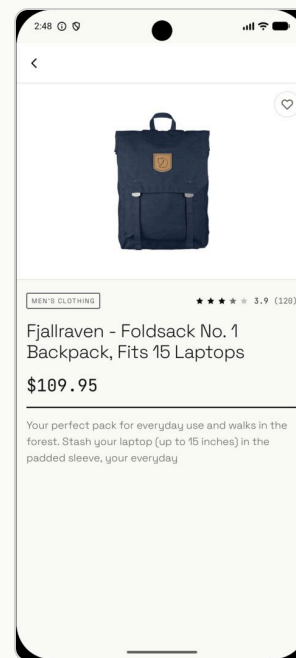
- 01 los tests de arquitectura mecánicos
- 02 los tests de interfaz instrumentados
- 03 la etapa dos completa

Decisión por decisión, con su fecha y su costo.

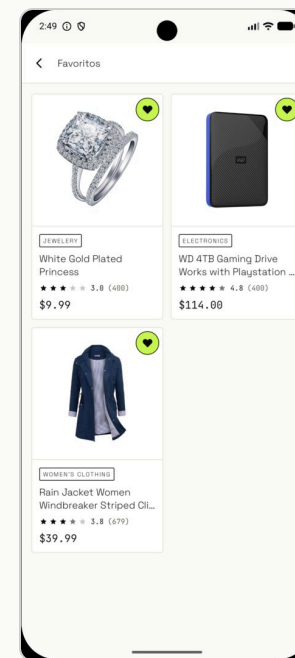
Gracias.



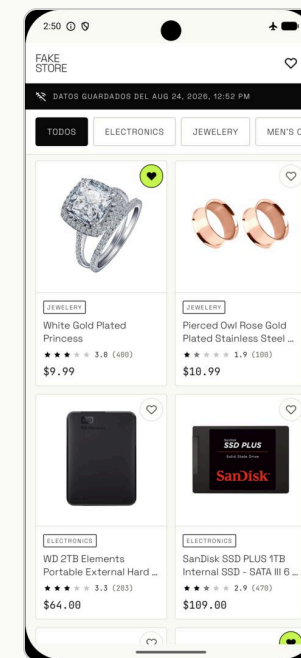
CATÁLOGO



DETALLE



FAVORITOS



OFFLINE